

A Small MNIST Neural Network from Scratch

1 Overview

This document describes the fully connected neural network implemented in this project. The goal is educational clarity: all matrix shapes, forward-pass values, loss calculations, and backpropagation steps are written explicitly.

The network classifies a flattened MNIST image. MNIST is the standard handwritten digit dataset introduced by LeCun and collaborators [5]. Each image is a 28×28 grayscale image, normalized so each pixel value lies in $[0, 1]$, then flattened into a vector

$$a_0 \in \mathbb{R}^{784}.$$

The target label is represented as a one-hot vector

$$y \in \mathbb{R}^{10},$$

where $y_k = 1$ for the correct digit class and all other entries are zero.

2 Architecture

The network has two hidden layers of 16 neurons each and an output layer of 10 neurons.

Quantity	Meaning	Shape	Activation
a_0	Input image	784	none
z_1	Hidden layer 1 pre-activation	16	none
a_1	Hidden layer 1 activation	16	ReLU
z_2	Hidden layer 2 pre-activation	16	none
a_2	Hidden layer 2 activation	16	ReLU
z_3	Output logits	10	none
a_3	Output probabilities	10	softmax

The learned parameters are

$$\begin{aligned} W_1 &\in \mathbb{R}^{16 \times 784}, & b_1 &\in \mathbb{R}^{16}, \\ W_2 &\in \mathbb{R}^{16 \times 16}, & b_2 &\in \mathbb{R}^{16}, \\ W_3 &\in \mathbb{R}^{10 \times 16}, & b_3 &\in \mathbb{R}^{10}. \end{aligned}$$

Weights are initialized with He initialization, which was derived for rectifier nonlinearities such as ReLU [3]. For a layer with *fan-in* n , the implementation samples each weight from a zero-mean normal distribution with standard deviation

$$\sigma = \sqrt{\frac{2}{n}}.$$

Equivalently, the variance is $2/n$. The factor of 2 is the important part: a ReLU sets negative pre-activations to zero, so under the rough symmetric-input assumption about half of the signal is removed. Scaling the variance by $2/n$ helps keep activation and gradient magnitudes from shrinking too quickly as they move through layers. This is the ReLU-specific analogue of variance-preserving initialization discussed in standard deep learning texts [2].

For this network, the layer-wise standard deviations are

$$\begin{aligned}\sigma_1 &= \sqrt{\frac{2}{784}}, \\ \sigma_2 &= \sqrt{\frac{2}{16}}, \\ \sigma_3 &= \sqrt{\frac{2}{16}}.\end{aligned}$$

Thus the code samples

$$\begin{aligned}(W_1)_{ij} &\sim \mathcal{N}(0, \sigma_1^2), \\ (W_2)_{ij} &\sim \mathcal{N}(0, \sigma_2^2), \\ (W_3)_{ij} &\sim \mathcal{N}(0, \sigma_3^2).\end{aligned}$$

All biases are initialized to zero.

3 Forward Pass

For one training example, the forward pass is

$$\begin{aligned}z_1 &= W_1 a_0 + b_1, \\ a_1 &= \text{ReLU}(z_1), \\ z_2 &= W_2 a_1 + b_2, \\ a_2 &= \text{ReLU}(z_2), \\ z_3 &= W_3 a_2 + b_3, \\ a_3 &= \text{softmax}(z_3).\end{aligned}$$

The ReLU activation is applied elementwise:

$$\text{ReLU}(x) = \max(0, x).$$

Its derivative is

$$\text{ReLU}'(x) = \begin{cases} 1, & x > 0, \\ 0, & x \leq 0. \end{cases}$$

ReLU is simple, but it is not just an arbitrary choice: rectifier nonlinearities are one reason He initialization is useful, because the initialization explicitly accounts for their zeroing behavior [3].

The softmax converts logits into class probabilities, a standard output choice for multiclass classification [1, 2]:

$$a_{3,i} = \frac{e^{z_{3,i}}}{\sum_{j=1}^{10} e^{z_{3,j}}}.$$

The implementation uses the numerically stable equivalent

$$a_{3,i} = \frac{e^{z_{3,i}-m}}{\sum_{j=1}^{10} e^{z_{3,j}-m}}, \quad m = \max_j z_{3,j}.$$

Subtracting m does not change the probabilities, but it reduces the chance of overflow during exponentiation.

4 Loss Function

The loss for one example is cross entropy, the usual negative log-likelihood loss for a categorical softmax model [1, 2]:

$$L(a_3, y) = - \sum_{i=1}^{10} y_i \log(a_{3,i}).$$

Because y is one-hot, this is equivalent to

$$L(a_3, y) = -\log(a_{3,k}),$$

where k is the correct class. The implementation clamps the value inside the logarithm with a small epsilon to avoid evaluating $\log(0)$.

5 Backpropagation

Backpropagation computes derivatives of the loss with respect to all weights and biases. It is an efficient application of the chain rule to computational graphs [6, 2]. The implementation accumulates gradients over a mini-batch before doing one stochastic gradient descent update.

5.1 Output Layer

For softmax followed by cross entropy, the derivative with respect to the logits z_3 simplifies to a particularly useful expression [1]:

$$\delta_3 = \frac{\partial L}{\partial z_3} = a_3 - y.$$

The shape is

$$\delta_3 \in \mathbb{R}^{10}.$$

Here is the short derivation. Let $p_i = a_{3,i}$ and let z_k mean the k th component of z_3 . The softmax derivative is

$$\frac{\partial p_i}{\partial z_k} = p_i(\mathbf{1}_{i=k} - p_k),$$

where $\mathbf{1}_{i=k}$ is 1 when $i = k$ and 0 otherwise. Since

$$L = - \sum_i y_i \log p_i,$$

we have

$$\begin{aligned}
\frac{\partial L}{\partial z_k} &= \sum_i \frac{\partial L}{\partial p_i} \frac{\partial p_i}{\partial z_k} \\
&= \sum_i \left(-\frac{y_i}{p_i} \right) p_i (\mathbf{1}_{i=k} - p_k) \\
&= -y_k + p_k \sum_i y_i.
\end{aligned}$$

For a one-hot target, $\sum_i y_i = 1$, so

$$\frac{\partial L}{\partial z_k} = p_k - y_k.$$

Stacking this result for all k gives $\delta_3 = a_3 - y$.

Since

$$z_3 = W_3 a_2 + b_3,$$

each output logit depends linearly on the weights in its row of W_3 . The parameter gradients are

$$\begin{aligned}
\frac{\partial L}{\partial W_3} &= \delta_3 a_2^T, \\
\frac{\partial L}{\partial b_3} &= \delta_3.
\end{aligned}$$

The matrix shape is

$$\delta_3 a_2^T \in \mathbb{R}^{10 \times 16}.$$

5.2 Second Hidden Layer

The gradient must pass backward through W_3 and through the ReLU activation at z_2 :

$$\delta_2 = \frac{\partial L}{\partial z_2} = (W_3^T \delta_3) \odot \text{ReLU}'(z_2),$$

where $\odot$ means elementwise multiplication. The shape is

$$\delta_2 \in \mathbb{R}^{16}.$$

This equation is just the chain rule in two stages. First, a_2 influences the loss through the output logits

$$z_3 = W_3 a_2 + b_3.$$

Because $\delta_3 = \partial L / \partial z_3$, the derivative of the loss with respect to the hidden activation a_2 is

$$\frac{\partial L}{\partial a_2} = W_3^T \delta_3.$$

The transpose appears because forward propagation multiplies by W_3 , so the backward pass multiplies by the linear map's transpose. In component form, for hidden unit j ,

$$\frac{\partial L}{\partial a_{2,j}} = \sum_{k=1}^{10} \frac{\partial L}{\partial z_{3,k}} \frac{\partial z_{3,k}}{\partial a_{2,j}} = \sum_{k=1}^{10} \delta_{3,k} (W_3)_{k,j}.$$

Second, a_2 is not an independent parameter; it is the ReLU of z_2 :

$$a_2 = \text{ReLU}(z_2).$$

So the derivative with respect to z_2 is obtained by multiplying elementwise by $\text{ReLU}'(z_2)$. If a unit's pre-activation $z_{2,j}$ was negative or zero, ReLU blocked that unit in the forward pass, and the local derivative sends no gradient through that unit.

The gradients for the second hidden layer parameters are

$$\begin{aligned}\frac{\partial L}{\partial W_2} &= \delta_2 a_1^T, \\ \frac{\partial L}{\partial b_2} &= \delta_2.\end{aligned}$$

The matrix shape is

$$\delta_2 a_1^T \in \mathbb{R}^{16 \times 16}.$$

5.3 First Hidden Layer

The gradient passes backward through W_2 and through the ReLU activation at z_1 :

$$\delta_1 = \frac{\partial L}{\partial z_1} = (W_2^T \delta_2) \odot \text{ReLU}'(z_1).$$

The shape is

$$\delta_1 \in \mathbb{R}^{16}.$$

The gradients for the first hidden layer parameters are

$$\begin{aligned}\frac{\partial L}{\partial W_1} &= \delta_1 a_0^T, \\ \frac{\partial L}{\partial b_1} &= \delta_1.\end{aligned}$$

The matrix shape is

$$\delta_1 a_0^T \in \mathbb{R}^{16 \times 784}.$$

6 Mini-Batch Gradient Accumulation

For a mini-batch B containing n examples, the code first zeros all gradient buffers:

$$dW_1, db_1, dW_2, db_2, dW_3, db_3 \leftarrow 0.$$

For each example $(a_0^{(r)}, y^{(r)})$ in the mini-batch, it runs a forward pass, computes the deltas, and accumulates

$$\begin{aligned}dW_3 &+= \delta_3^{(r)} (a_2^{(r)})^T, & db_3 &+= \delta_3^{(r)}, \\ dW_2 &+= \delta_2^{(r)} (a_1^{(r)})^T, & db_2 &+= \delta_2^{(r)}, \\ dW_1 &+= \delta_1^{(r)} (a_0^{(r)})^T, & db_1 &+= \delta_1^{(r)}.\end{aligned}$$

After all examples in the batch have contributed, the update uses the average gradient:

$$\begin{aligned} W_1 &\leftarrow W_1 - \eta \frac{dW_1}{n}, & b_1 &\leftarrow b_1 - \eta \frac{db_1}{n}, \\ W_2 &\leftarrow W_2 - \eta \frac{dW_2}{n}, & b_2 &\leftarrow b_2 - \eta \frac{db_2}{n}, \\ W_3 &\leftarrow W_3 - \eta \frac{dW_3}{n}, & b_3 &\leftarrow b_3 - \eta \frac{db_3}{n}, \end{aligned}$$

where η is the learning rate. Mini-batch stochastic gradient descent is a standard optimization method for neural networks [2].

7 Training Loop

The training loop shuffles the training examples at the start of each epoch, then processes the shuffled data in mini-batches. Each mini-batch follows this sequence:

1. Set all accumulated gradients to zero.
2. For each example in the mini-batch, run forward propagation and backpropagation, adding into the gradient buffers.
3. Update weights and biases with SGD using the average batch gradient.

After each epoch, the program reports average training loss, training accuracy, and test accuracy.

8 Comparison with the PyTorch Version

The repository also contains a PyTorch implementation in `pytorch_version`. It solves the same MNIST classification task, but it is not the same model or the same training setup. The C++ implementation is intentionally small and explicit so that the forward pass and backpropagation equations can be inspected directly. The PyTorch implementation is a more practical baseline that relies on framework components for automatic differentiation, optimized tensor operations, and common training defaults.

Feature	C++ educational version	PyTorch version
Architecture	784 $\rightarrow$ 16 $\rightarrow$ 16 $\rightarrow$ 10	784 $\rightarrow$ 128 $\rightarrow$ 10
Hidden activations	ReLU, ReLU	ReLU
Approx. parameters	13,002	101,770
Output handling	explicit softmax	raw logits
Loss	explicit cross entropy	<code>nn.CrossEntropyLoss</code>
Optimizer	mini-batch SGD	Adam
Input scaling	pixels in [0, 1]	normalized by MNIST mean/std
Backpropagation	handwritten	PyTorch autograd

The most visible architectural difference is width. The C++ network has two hidden layers, but each has only 16 neurons. Its parameter count is

$$16 \cdot 784 + 16 + 16 \cdot 16 + 16 + 10 \cdot 16 + 10 = 13,002.$$

The PyTorch network has one hidden layer with 128 neurons:

$$128 \cdot 784 + 128 + 10 \cdot 128 + 10 = 101,770.$$

So the PyTorch model has about 7.8 times as many trainable parameters. That extra capacity gives it more ways to represent digit shapes and usually improves accuracy on MNIST.

The loss functions are closer mathematically than they first appear. In the C++ version, the model computes

$$a_3 = \text{softmax}(z_3)$$

and then evaluates cross entropy. In PyTorch, `nn.CrossEntropyLoss` expects raw logits and internally applies a numerically stable log-softmax followed by negative log likelihood. Thus both versions optimize the same basic softmax classification objective [1, 2]. The PyTorch form is usually preferred in production code because combining log-softmax and cross entropy avoids explicitly forming probabilities that may underflow or round to zero.

The optimization setup is also different. The C++ version uses plain mini-batch stochastic gradient descent so that the parameter update is easy to see:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L.$$

The PyTorch version uses Adam, an adaptive optimizer that keeps moving averages of gradients and squared gradients for each parameter [4]. Adam often reaches a good solution with less learning-rate tuning than plain SGD, which helps explain why the PyTorch baseline converges quickly.

Finally, the two programs preprocess inputs differently. The C++ loader scales pixels from integer values $0, \dots, 255$ into $[0, 1]$. The PyTorch script first converts to tensors and then normalizes using the MNIST training-set mean and standard deviation:

$$x \leftarrow \frac{x - 0.1307}{0.3081}.$$

Centering and scaling inputs often improves conditioning for gradient-based optimization [2].

In short, the C++ version is designed to make the math transparent. The PyTorch version performs better because it is wider, uses Adam, uses stable built-in cross entropy on logits, and normalizes the inputs more aggressively. A fairer performance comparison would give both implementations the same architecture, preprocessing, optimizer, initialization, batch size, and number of epochs.

9 Shape Summary

Expression	Shape calculation	Result
$W_1 a_0 + b_1$	$(16 \times 784)(784) + (16)$	16
$W_2 a_1 + b_2$	$(16 \times 16)(16) + (16)$	16
$W_3 a_2 + b_3$	$(10 \times 16)(16) + (10)$	10
$\delta_3 a_2^T$	$(10)(1 \times 16)$	10×16
$\delta_2 a_1^T$	$(16)(1 \times 16)$	16×16
$\delta_1 a_0^T$	$(16)(1 \times 784)$	16×784

10 Connection to the Code

The implementation keeps the same names as this document: `a0`, `z1`, `a1`, `z2`, `a2`, `z3`, and `a3`. These values are cached during the forward pass because the backward pass needs them to compute the derivatives above.

The important methods are:

- `forward(input)` computes and caches all activations.
- `backward(input, target)` accumulates one example's gradients.
- `zeroGradients()` clears the mini-batch gradient buffers.
- `updateParameters(learningRate, batchSize)` applies averaged SGD updates.
- `computeLoss(output, target)` evaluates cross entropy.
- `predict(input)` returns the index of the largest softmax probability.

References

- [1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Available at <https://www.deeplearningbook.org/>.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In *Proceedings of the IEEE International Conference on Computer Vision*, 2015. <https://arxiv.org/abs/1502.01852>.
- [4] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. <https://arxiv.org/abs/1412.6980>.
- [5] Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [6] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.